VOTE BANKER

Backend Architecture, API Map & Hosting Plan

Current implementation review | Node.js 22+ / Express 4 / Supabase

Area	Current choice
Runtime	Node.js 22 or newer
HTTP API	Express 4 with JSON, CORS, Helmet, and rate limiting
Authentication	Twilio Verify OTP + Supabase Auth sessions; OAuth integrations
Database and storage	Supabase PostgreSQL, Supabase Auth, and Supabase Storage for profile photos
Current production hosting	Railway backend; Vercel Flutter web frontend
Recommended target	Keep Railway initially, add health checks, observability, backups, and a managed scaling plan

Executive recommendation: the current Railway + Supabase + Vercel architecture is a good fit for this stage. It is simple, fast to deploy, and avoids operating a database cluster. Before a large user launch, add production environment validation, structured logging, monitoring, automated database backups, and a second backend replica or an autoscaling plan.

1. System architecture

The Flutter web/mobile client calls one stateless Express API. The API validates requests, authenticates the user, applies business rules, and talks to managed external services. Secrets stay in the backend environment and never ship in Flutter.

Request flow

Layer	Responsibility	Technology in this repository
Client	Screens, forms, session storage, API calls	Flutter; HTTP package; API_BASE_URL
Edge / hosting	TLS, public URL, process lifecycle, deploys	Railway for API; Vercel for web frontend
HTTP middleware	Security headers, CORS, JSON parsing, global API limit	Helmet; cors; express.json; express-rate-limit
Routes	HTTP methods, validation, auth guard, response mapping	src/routes/*.js plus domain routers
Domain services	Auth, AI, social, domains, OCR, merchandise, volunteers	src/*.js and src/vrm/*.js
Data and identity	Users, RLS-protected tables, photos, sessions	Supabase Auth, PostgreSQL, Storage
External providers	OTP, OAuth, domain availability, AI generation	Twilio, Meta/Google/X, Openprovider, OpenAI-compatible API/Ollama

Logical diagram

Flutter App	HTTPS JSON API	Express Backend
Web / Android / iOS	Authorization: Bearer | CORS	Helmet → CORS → JSON → rate limit → routers
Profile, website, letters, speech, VRM, volunteers	↕	Auth guards → domain services → adapters
	■ ↓ ■	
	Supabase	Twilio / OAuth / AI / Openprovider

The backend is intentionally stateless except for short-lived in-memory social OAuth tickets. For multiple replicas, move that ticket store to Redis or another shared store so a callback can land on any replica.

2. Backend source structure

Path	Purpose
src/server.js	Express application bootstrap, middleware, health endpoint, route mounting, 404/error handling, server lifecycle
src/env.js	Loads local .env values without overwriting already-defined process variables
src/lib/clients.js	Lazy Twilio and Supabase public/admin client creation; required secret checks
src/routes/auth.js	OTP sign-in, Supabase session creation, social OAuth start/callback/exchange
src/routes/profile.js	Authenticated profile read/write and profile photo upload/delete
src/routes/websites.js	Website draft, AI generation, and publish/payment gate
src/routes/locations.js	Country, state, district, and assembly constituency lookups
src/merchandise.js	Product and order API
src/speeches.js / src/speech_ai.js	Speech generation and AI provider configuration
src/letters.js / src/letter_ai.js / src/letter_compose.js	Official letter generation with AI or formal fallback composition
src/domains.js / src/openprovider.js	Domain search and Openprovider integration
src/routes/vrm.js / src/vrm/*	Voter-record import, OCR parsing, contacts, and shared catalog access
src/routes/social_connections.js / social_auth.js	Connected social accounts and OAuth token flows
src/routes/social_media.js	Social contacts, integrations, posts, and captions
src/volunteers.js / src/team_members.js	Volunteer management and team member CRUD APIs
supabase/migrations/*	Versioned PostgreSQL schema, RLS, indexes, and storage-related changes

3. API and routing map

Mount	Methods / endpoints	Auth / purpose
/api/health	GET /	Public liveness check
/api/auth	POST /otp/send, POST /otp/verify; GET /oauth/:provider/start; GET /oauth/callback; POST /oauth/exchange	Public start points; OTP and OAuth are rate-limited
/api/profile	GET /, PUT /, POST /photo, DELETE /photo	Authenticated; profile and photo
/api/websites	POST /generate; GET /me; PUT /me; POST /me/publish	Generate is limited; draft/publish authenticated
/api/location	GET /countries, /states, /districts, /assembly-constituencies	Public catalog lookups
/api/merchandise	GET /products; GET|POST /orders	Products public; orders user-scoped
/api/speeches	POST /generate	Generation endpoint; provider-backed or configured fallback
/api/letters	POST /generate	Generation endpoint; provider-backed or formal composer
/api/domains	POST /search; GET /diagnostics	Openprovider credentials server-side
/api/vrm	GET /records; PATCH /records/:id/contacts; POST /import/preview-ocr; POST /import/confirm; POST /import/preview-pdf	Mixed optional/public user access; import is protected and rate-limited
/api/social-connections	GET /; POST /:provider/start; DELETE /:provider	Authenticated OAuth connections

Mount	Methods / endpoints	Auth / purpose
/api/social-media	GET /contacts; GET /integrations; POST /posts; POST /captions	Authenticated
/api/volunteers	GET /stats; GET /overview; GET|POST /volunteers; GET|PATCH|DELETE /volunteers/:id; GET|POST /tasks; PATCH /tasks/:id; GET|POST /applications; POST /applications/:id/review; GET|POST /teams	User-scoped management APIs
/api/team-members	GET|POST /; GET|PATCH|DELETE /:id	Team member management

4. Authentication, authorization, and data flow

Phone OTP flow

- Client sends phone number and channel to POST /api/auth/otp/send.
- Backend normalizes and validates E.164 phone format, then asks Twilio Verify to send the code.
- Client sends phone and code to POST /api/auth/otp/verify.
- After Twilio verification, the server derives a phone-based Supabase identity using an HMAC keyed by SUPABASE_SECRET_KEY.
- Server signs in or creates the Supabase user and returns access_token and refresh_token.
- Subsequent protected calls use Authorization: Bearer ; the backend validates the user with Supabase.

OAuth flow

- Provider start endpoint signs a short-lived state value containing provider, return URL, expiry, nonce, and purpose.
- Provider redirects to the backend callback; backend exchanges the provider code server-side.
- The backend creates/signs in the mapped Supabase user and returns a one-time ticket for the frontend to exchange.
- For multiple backend replicas, replace the current in-memory ticket store with Redis.

Data boundary

Data	System of record	Protection
Users and sessions	Supabase Auth	Supabase tokens; server secret key never exposed
Profiles, websites, VRM, volunteers, team members, connections	Supabase PostgreSQL	User scoping plus RLS migrations; server checks
Profile photos	Supabase Storage	Server upload/delete routes; public URL mapping as configured
OTP codes	Twilio Verify	Never store codes in this backend
Social tokens	Supabase tables, encrypted/configured by SOCIAL_TOKEN_KEY	Keep encryption key in secrets; never log tokens
AI and domain credentials	Backend environment	Never place in Flutter or browser code

5. Exact platform and API components

Concern	Component	Why it is used
Language/runtime	JavaScript on Node.js 22+	Matches package engine and current source
Web framework	Express 4.22	Routing, middleware, JSON APIs, familiar deployment model
Security headers	Helmet 8	Baseline browser security headers
Cross-origin policy	cors 2.8	Allows configured production frontend and local development origins
Rate limiting	express-rate-limit 8	Global API limit plus tighter OTP/import/generation limits
Configuration	dotenv plus src/env.js	Local .env and platform environment variables
HTTP API format	REST-style HTTPS JSON	Simple Flutter integration; status codes and JSON error objects
Database client	@supabase/supabase-js	Auth and Supabase admin/public operations
SQL driver	pg	Available for direct PostgreSQL access where needed
OTP	Twilio Verify	Managed SMS verification; WhatsApp can be enabled after setup
OAuth	Meta, Google, and X provider flows	Social sign-in and connected accounts
Domain search	Openprovider REST API	Availability checks and TLD catalog
AI	OpenAI-compatible /v1 API or local Ollama	Speech, letter, website generation; configurable model/base URL
OCR	Server-side PDF/image OCR utilities	VRM electoral-roll import; optional and feature-flagged
Testing	Node built-in test runner	Backend tests under test/

6. Environment variables and secret ownership

Use Railway Variables (or an equivalent secret manager) in production. Do not copy production secrets into the Flutter project, source control, screenshots, issue trackers, or chat.

Group	Variables
Process / browser access	PORT, CORS_ORIGINS, PUBLIC_API_URL, FRONTEND_URL
Supabase	SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, SUPABASE_SECRET_KEY, DATABASE_URL
OTP	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_VERIFY_SERVICE_SID, WHATSAPP_OTP_ENABLED
Social OAuth	META_APP_ID, META_APP_SECRET, INSTAGRAM_APP_ID, INSTAGRAM_APP_SECRET, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, X_CLIENT_ID, X_CLIENT_SECRET, WHATSAPP_CONFIG_ID
AI	SPEECH_AI_API_KEY, SPEECH_AI_BASE_URL, SPEECH_AI_MODEL, WEBSITE_AI_API_KEY, WEBSITE_AI_BASE_URL, WEBSITE_AI_MODEL
Domains	OPENPROVIDER_USERNAME, OPENPROVIDER_PASSWORD, OPENPROVIDER_BASE_URL, OPENPROVIDER_TIMEOUT_MS, OPENPROVIDER_MAX_EXTENSIONS, OPENPROVIDER_BATCH_SIZE, OPENPROVIDER_CHECK_CONCURRENCY
VRM/OCR	VRM_PDF_PATH, VRM_ALLOW_SERVER_PDF_IMPORT, optional TESSERACT_PATH/PDFTOPPM_PATH/POPPLER_BIN_DIR
Token encryption	SOCIAL_TOKEN_KEY (fallback is SUPABASE_SECRET_KEY)

7. Hosting plan for smooth and fast operation

Recommended now: Railway + Supabase + Vercel

Service	Recommendation	Operational settings
Backend	Railway service running npm start	Node 22; PORT from platform; health check /api/health; region close to primary users and Supabase ap-south-1
Database/Auth	Supabase managed project	Keep database and API region aligned; enable backups; review RLS and indexes before launch
Frontend	Vercel Flutter web deployment	Set API_BASE_URL to the Railway HTTPS URL; configure production CORS_ORIGINS
Files	Supabase Storage	Use size/type limits; serve through controlled policies; avoid large base64 payloads
Observability	Railway logs plus an error/metrics service	Add request IDs, structured JSON logs, latency, 4xx/5xx, rate-limit, provider failures

Why Railway is a good fit

- The repository already documents Railway production hosting and deployment from main.
- Node/Express deployment is direct; no container orchestration is required for the initial launch.
- Environment variables and logs are available in one service dashboard.
- Scaling can start with one instance and move to multiple replicas when traffic requires it.

When to upgrade the hosting design

Signal	Upgrade
Frequent cold starts or slow first request	Use an always-on instance and warm-up health checks
Sustained CPU/memory pressure	Increase Railway resources or add replicas behind the platform load balancer
OAuth callback failures across replicas	Move social ticket storage from memory to Redis
AI/OCR jobs block requests	Move long jobs to a queue/worker service and return job status
Large electoral-roll imports	Use object storage plus asynchronous processing; do not keep large PDFs in request memory
Regional latency outside India	Add a region closer to users or use a global edge/API strategy; keep database writes near Supabase
Strict enterprise compliance	Consider AWS ECS/Fargate or GCP Cloud Run with managed secrets, logs, WAF, and private networking

8. Production readiness checklist

Priority	Action	Reason
P0	Populate all required Railway secrets; verify no secrets are committed	Auth and integrations fail without them; secret leakage is high risk
P0	Set CORS_ORIGINS to the exact production frontend URL	Prevent unintended browser origins
P0	Keep SUPABASE_SECRET_KEY, Twilio, OAuth, AI, and Openprovider credentials server-only	These keys can access or spend money
P0	Run database migrations and verify RLS policies for every user-owned table	Prevent cross-user data access
P0	Configure /api/health as Railway health check and set restart policy	Fast detection and recovery
P1	Add request ID, structured logs, latency metrics, and alerting	Diagnose production failures quickly
P1	Add timeouts and retry policy per external provider	Avoid hung requests and retry storms
P1	Move social OAuth tickets to Redis before enabling replicas	In-memory tickets are process-local
P1	Add pagination and indexes to volunteer, team, VRM, and social-contact queries	Keep response time stable as data grows
P1	Add automated Supabase backups and a restore drill	Protect user and operational data
P2	Move OCR and AI generation to background jobs when usage grows	Prevent long requests from consuming API workers
P2	Add API contract/OpenAPI documentation and integration tests	Make frontend and backend changes safer
P2	Add dependency audit and lockfile review to CI	Reduce supply-chain risk

9. Local and production commands

Local backend

```
cd VoteBanker_Backend-main
npm install
cp .env.example .env
npm run dev
```

Production backend

```
npm ci
npm start
```

Health check

```
GET /api/health → { "status": "ok", "service": "VOTE BANKER backend" }
```

Bottom line: deploy the current backend to Railway in the same broad region as the Supabase project, keep the frontend on Vercel, and invest first in secrets, RLS verification, health checks, observability, and provider timeouts. This gives the best balance of speed, cost, and operational simplicity for the current application.